

SignalDesk — a news ranking engine that shows its work

A Telegram bot that reads 12 crypto and macro feeds, ranks every story with an auditable eight-factor score, and lets a language model write only what the algorithm already chose. Polymer Capital Tech Expo 2026.

PROBLEM STATEMENT

Twelve newsfeeds produce a few hundred crypto and macro headlines a day — mostly listicles, price-prediction bait, and the same story told six times by six outlets. The signal is in there; the cost of finding it is forty minutes every morning, and the reader who most needs a fast multi-angle view of the market is exactly the one who cannot spend it. The obvious fix — hand the pile to a language model — fails in a specific way. **The model's selection is unauditable.** When it leads with the wrong story you cannot find out why, cannot reproduce it, and have nothing to correct. Forty minutes of reading has been traded for a black box.

SOLUTION OVERVIEW

Split the job at its natural seam. **An algorithm decides what matters** — deterministic, inspectable, covered by 148 offline tests. **A model writes it up** — it receives the already-ranked list and is told in its system prompt that selection is not its job: it cannot re-order, add stories, or introduce a number absent from the input. Three commands: `/digest` (the briefing, pushed automatically at 08:30), `/top` (the live ranking with scores), and `/weights` (five subject filters plus a numbers-vs-analysis depth setting that visibly moves two weights in the formula). Between briefings it re-checks every 15 minutes and pushes anything above threshold at once.

- 0 • **hard filters**
clickbait, price ticks, off-topic macro — binary
- 1 • **dedupe**
TF-IDF cosine + rare-entity overlap, transitive closure
- 2 • **eight-factor score**
keyword ·185 recency ·185 source ·185 topicality ·130 numeric ·100 analysis ·100 corroboration ·075 asset ·040
- 3 • **adaptive top-N**
quality gate, per-subject diversity cap, count as a floor; crypto and macro in separate pools
- 4 • **LLM write-up → Telegram**
08:30 digest, threshold alerts, live commands

```
/why 1 - "Curve exploited for $62m"
```

factor	raw	wt	adds
keyword	1.00	0.185	0.185
source_quality	0.95	0.185	0.176
recency	0.94	0.185	0.173
topicality	0.50	0.130	0.065
numeric	0.35	0.100	0.035
asset	0.50	0.040	0.020
analysis	0.00	0.100	0.000
source_count	0.00	0.075	0.000
TOTAL			0.654

USE OF AI

Built with Claude Code as a pair programmer throughout — scaffolding, the scoring module, and every test in the suite. **In the product:** Moonshot **Kimi K3** with xAI **Grok 4.3** as fallback, through TokenRouter's OpenAI-compatible endpoint — cross-vendor on purpose, because two models from one provider are a retry, not a fallback. **Free, unauthenticated data:** 12 RSS feeds — CoinDesk, The Block, DL News and Unchained on crypto; FT, Bloomberg, CNBC and the ECB press feed on macro — plus the **Binance** 24h ticker and the **alternative.me Fear & Greed** index. The system prompt enforces the boundary:

“You will be given a numbered list of stories that has ALREADY been selected and ranked by a scoring engine. **Selection is not your job.** Do not re-order them... **Never invent a number.** Prices, percentages, and dollar amounts may only appear if they appear in the input... The headline **MUST** be about story 1 — choosing a different story is a selection decision.”

When every provider fails, the bot emits template output from the top-ranked headlines verbatim — degraded, honest, still correctly ordered: the ranking never depended on the model.

IMPACT & VALUE

Forty minutes of triage becomes a thirty-second read, and — unlike a summariser — the output can be challenged: `/why n` answers “why is this at the top” with arithmetic, so a wrong lead story traces to a factor weight, not to a sampling temperature. Marginal cost is one LLM call per digest and the data layer is free, so it costs nothing to scale — which makes the next step viable: a **free subscription site** where readers pick their subjects and receive the same push, free tier as the growth channel rather than a loss leader.

REFLECTIONS

Making the score explain itself changed how I built the whole thing. `/why` began as a nice-to-have and became the debugging tool — **every real bug here was found by reading the factor table beside a ranking, and not one was caught by a test.** A `\b(exploit)\b` pattern could not match “*exploited*”, so a \$62m protocol hack scored 0.00 on the heaviest factor and ranked 7th of 8; after the fix, 1.00 and 2nd. Three later bugs shared one habit — adding a broad word to a category and checking only that the intended headline matched — fixed by bidirectional tests.

Given more time: the eight weights are **reasoned, not fitted** — every one defensible, none measured. Logging each ranking against which stories readers open would turn eight arguments into eight numbers. The classifier is regex over open-ended language, so it will keep having gaps: maintained, never finished.